

Section 5

Week 6 — Indexing I: Basics + EXPLAIN

Goal: Understand what indexes are, how they work internally, and how to use EXPLAIN / EXPLAIN ANALYZE to see what PostgreSQL is actually doing with your queries.

Today's Agenda

- 1. **What is an Index** — the phone book analogy + B-Tree structure
- 2. **The Read/Write Trade-off** — why indexes aren't free
- 3. **EXPLAIN vs EXPLAIN ANALYZE** — reading query plans
- 4. **Seq Scan vs Index Scan** — seeing the difference live
- 5. **Single vs Composite Indexes** — column order matters
- 6. **Student Lab**

1. Bridge from Week 5

In the last two weeks we dealt with *correctness*:

- Week 4: isolation levels → prevent bad reads
- Week 5: locks → prevent bad writes (lost updates, deadlocks)

Today we shift to **speed**. A query can be completely correct and still take 10 seconds on a large table. The tool that fixes this is the **index**.

2. What Is an Index?

An index is a **separate data structure** that PostgreSQL maintains alongside your table. It lets the database find rows much faster — without scanning every row.

Analogy — phone book

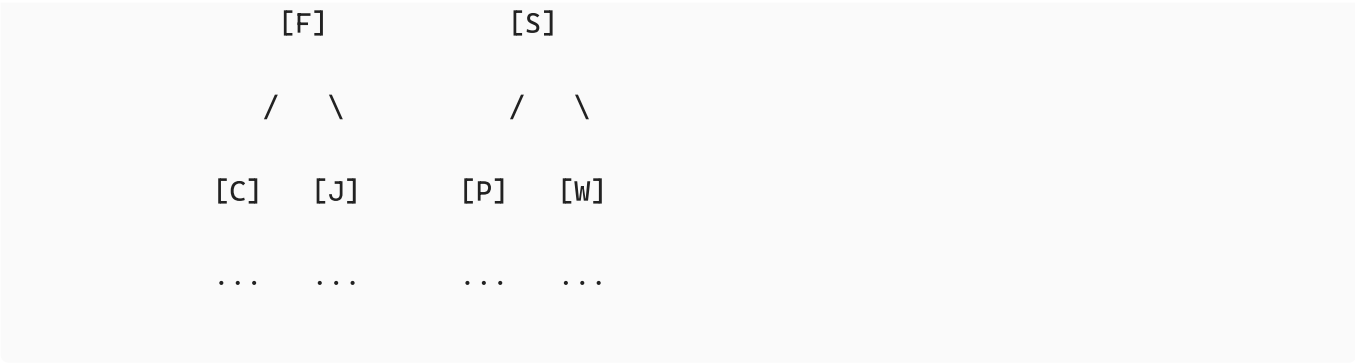
Without index: to find "Shady", you read every name from A to Z until you find it. That's a **Sequential Scan (Seq Scan)** — slow on large tables.

With index: you open the phone book at the right letter, jump directly to the page. That's an **Index Scan** — fast regardless of table size.

What PostgreSQL's default index type is

PostgreSQL uses **B-Tree** (Balanced Tree) by default. It's a sorted tree structure where each level splits the data in half.





B-Tree with numbers — how it actually works

The same idea applies to numbers, which is exactly what you see with `customer_id`.

Imagine the `rental` table has these `customer_id` values:

Rows in table (unordered on disk):

148, 391, 23, 507, 76, 290, 412, 11, 200, 350 ...

B-Tree index (PostgreSQL keeps this sorted automatically):

[200]

/ \

[76]

[350]

/ \ / \

[23]

[148]

[290]

[412]

/ \ / \ / \

[11] .. [76] .. [290] .. [391] [507]

Lookup: `WHERE customer_id = 148`

Without index → scan every row until 148 is found.

With index → the B-Tree guides you in 3 steps:

Start at root: 148 < 200 → go LEFT

At node [76]: 148 > 76 → go RIGHT

At node [148]: found!  → fetch the actual row from disk

3 steps instead of reading thousands of rows.

Range query: `WHERE customer_id > 200`

The B-Tree is sorted, so PostgreSQL finds 200 and reads everything to its right in order:

```
200 → 290 → 350 → 391 → 412 → 507 → ...
```

Nothing below 200 is touched. This is why indexes work for `>`, `<`, `BETWEEN`.

Why `LIKE '%abc'` breaks it

The tree is sorted left-to-right. `LIKE 'ABC%'` works — PostgreSQL knows where 'ABC' starts and scans forward. But `LIKE '%abc'` could match anything at any position — the tree can't narrow it down. Full scan required.

- Every lookup: $O(\log n)$ steps — fast even for millions of rows
- Works for: `=`, `<`, `>`, `BETWEEN`, `ORDER BY`, `LIKE 'abc%'`
- Does NOT help: `LIKE '%abc'` (leading wildcard)

3. Why Indexes Speed Reads but Slow Writes

Every write now has **two jobs**: update the table row AND update the index.

INSERT — two writes happen

```
-- Index on rental(customer_id) currently contains: ...[148]...[200]...[290]...

INSERT INTO rental (rental_date, inventory_id, customer_id, staff_id, return_date)
VALUES (NOW(), 367, 175, 1, NULL);
```

What PostgreSQL does internally:

1. Write the new row to the rental table ← always happens
2. Insert `customer_id = 175` into the B-Tree index ← extra work

Index before: ...[148]...[200]...

Index after: ...[148]...[175]...[200]... ← 175 inserted in sorted order

UPDATE — depends on which column changes

```
-- Case 1: updating the indexed column (customer_id)

UPDATE rental SET customer_id = 175 WHERE rental_id = 1001;
```

1. Update the row in the table

- 2. Remove old entry [148] from the B-Tree
- 3. Insert new entry [175] into the B-Tree (correct position)

Index before: ...[100]...[148]...[200]...

Index after: ...[100]...[175]...[200]...

```
-- Case 2: updating a column that is NOT indexed (return_date)

UPDATE rental SET return_date = NOW() WHERE rental_id = 1001;
```

- 1. Update the row in the table
- 2. Index is NOT touched – no extra work ← cheaper!

If you update a column that is not indexed, the index is not touched at all.

DELETE — index entry is removed

```
DELETE FROM rental WHERE rental_id = 1001; -- this row had customer_id = 148
```

- 1. Remove the row from the table
- 2. Remove entry [148] from the B-Tree index

Index before: ...[100]...[148]...[200]...

Index after: ...[100]...[200]...

Summary

Operation	Table	Index
SELECT	Read rows	Lookup only — no change
INSERT	+1 row	+1 entry inserted in sorted order
UPDATE (indexed col)	Update row	Remove old entry, insert new
UPDATE (other col)	Update row	Nothing — untouched
DELETE	Remove row	Remove entry

Rule of thumb: Index columns you frequently filter or join on, not every column. Over-indexing slows down writes and wastes disk space.

4. EXPLAIN vs EXPLAIN ANALYZE

These are your most important performance tools.

EXPLAIN — shows the plan, does NOT run the query

```
EXPLAIN SELECT * FROM rental WHERE customer_id = 148;
```

Output:

```
Seq Scan on rental (cost=0.00..310.44 rows=30 width=36)

  Filter: (customer_id = 148)
```

How to read it:

- Seq Scan → no index, reading all rows
- cost=0.00..310.44 → planner estimate (not milliseconds — arbitrary units)
- rows=30 → estimated result rows
- width=36 → estimated row size in bytes

The cost numbers are only useful for **comparison** — higher means slower.

EXPLAIN ANALYZE — runs the query, shows real timings

```
EXPLAIN ANALYZE SELECT * FROM rental WHERE customer_id = 148;
```

Output:

```
Seq Scan on rental (cost=0.00..310.44 rows=30 width=36)

      (actual time=0.021..1.842 ms rows=30 loops=1)

  Filter: (customer_id = 148)

  Rows Removed by Filter: 16014

Planning Time: 0.077 ms

Execution Time: 1.874 ms
```

New fields:

- actual time=0.021..1.842 ms → real startup → total time in milliseconds
- rows=30 loops=1 → actually returned 30 rows, ran once
- Rows Removed by Filter: 16014 → scanned 16,014 rows, threw them away to find 30
- Execution Time → total real runtime

⚠ **EXPLAIN ANALYZE runs the query.** Wrap UPDATE / DELETE checks in BEGIN / ROLLBACK to avoid side effects.

How to read cost numbers

cost=startup..total

- **startup cost**: before the first row is returned (e.g. sorting, index fetch setup)
- **total cost**: time to return all rows
- These are **not milliseconds** — they're internal PostgreSQL units. Compare them relative to each other: 300 vs 5 is a big difference.

Try it now

```
-- Run this - observe the scan type and Execution Time

EXPLAIN ANALYZE

SELECT *

FROM rental

WHERE customer_id = 148;
```

What to notice: Seq Scan , Rows Removed by Filter , and Execution Time . Keep this result — we'll compare it after adding an index.

5. Seq Scan vs Index Scan

Seq Scan (Sequential Scan)

PostgreSQL reads **every row** from disk. Happens when there is no usable index.

```
-- No index on customer_id yet

EXPLAIN ANALYZE

SELECT * FROM rental WHERE customer_id = 148;
```

```
Seq Scan on rental (cost=0.00..310.44 rows=30 width=36)

(actual time=0.021..1.842 ms rows=30 loops=1)

Filter: (customer_id = 148)

Rows Removed by Filter: 16014      ← scanned 16,014 to find 30

Execution Time: 1.874 ms
```

It scanned all 16,044 rows to return 30. Rows Removed by Filter: 16014 shows the waste.

After adding the index

```
-- Create the index
```

```
CREATE INDEX idx_rental_customer_id ON rental (customer_id);
```

```
-- Same query again
```

```
EXPLAIN ANALYZE
```

```
SELECT * FROM rental WHERE customer_id = 148;
```

```
Bitmap Heap Scan on rental (cost=4.64..103.25 rows=46 width=36)
```

```
(actual time=0.048..0.074 rows=46 loops=1)
```

```
Recheck Cond: (customer_id = 148)
```

```
Heap Blocks: exact=39
```

```
Buffers: shared hit=39 read=2
```

```
-> Bitmap Index Scan on idx_rental_customer_id
```

```
Index Cond: (customer_id = 148)
```

```
Planning Time: 0.912 ms
```

```
Execution Time: 0.087 ms
```

The scan type changed and `Rows Removed by Filter` is gone. The key number:

Execution Time: 1.874 ms → 0.087 ms (~20× faster)

The exact name of the scan (`Bitmap Heap Scan`, `Index Scan`) depends on how many rows match — PostgreSQL picks automatically. Don't worry about it. What matters: **Seq Scan is gone, and it's fast.**

When PostgreSQL ignores your index

If the table is very small (a few hundred rows), a Seq Scan can be faster — the overhead of the index isn't worth it. PostgreSQL decides this automatically.

Showing Difference

Step 1 — Create a demo table

```
CREATE TABLE students (  
  id SERIAL PRIMARY KEY,  
  grade CHAR(1)  
);
```

Step 2 — Insert many rows

We insert **100,000 students**.

Distribution:

- many **C**
- medium **B**
- few **A**

```
INSERT INTO students (grade)
SELECT
CASE
    WHEN random() < 0.7 THEN 'C'
    WHEN random() < 0.9 THEN 'B'
    ELSE 'A'
END
FROM generate_series(1,100000);
```

So approximately:

Grade	Rows
C	~70,000
B	~20,000
A	~10,000

Step 3 — Create an index

```
CREATE INDEX idx_students_grade
ON students(grade);
```

Step 4 — Rare value → Index Scan

```
EXPLAIN ANALYZE
SELECT *
FROM students
WHERE grade = 'A';
```

Expected plan:

```
Index Scan using idx_students_grade
```

Explain to students:

Only few rows match, so the index is useful.

Step 5 — Medium amount → Bitmap Scan

```
EXPLAIN ANALYZE
SELECT *
```



```
FROM students
WHERE grade = 'B';
```

Expected plan:

```
Bitmap Index Scan
Bitmap Heap Scan
```

Explain:

Too many rows for index jumps, but not enough for full table scan.

Step 6 — Many rows → Sequential Scan

```
EXPLAIN ANALYZE
SELECT *
FROM students
WHERE grade = 'C';
```

Expected:

```
Seq Scan on students
```

Explain:

Since most rows match, reading the whole table once is cheaper.

One powerful sentence to say in class

PostgreSQL doesn't use indexes because they exist — it uses them only when they reduce the cost of reading data.

Clean up

```
DROP INDEX IF EXISTS idx_rental_customer_id;
```

6. Creating Indexes

Basic syntax

```
CREATE INDEX idx_rental_customer_id ON rental (customer_id);
```

Create it, then run EXPLAIN ANALYZE to confirm the plan changed:

```
EXPLAIN ANALYZE

SELECT * FROM rental WHERE customer_id = 148;
```

```
-- Before index: Seq Scan, ~1.8 ms

-- After index:  Index Scan, ~0.09 ms
```

Drop an index

```
DROP INDEX idx_rental_customer_id;
```

See all indexes on a table

```
SELECT indexname, indexdef

FROM pg_indexes

WHERE tablename = 'rental';
```

Run this now on the DVD Rental database — you'll see which columns are already indexed by default (PostgreSQL creates indexes automatically on primary keys and unique constraints).

7. Single vs Composite Indexes

Single-column index

Good when your most common filter is on one column.

```
CREATE INDEX idx_rental_customer_id ON rental (customer_id);
```

```
-- This uses the index

EXPLAIN ANALYZE

SELECT * FROM rental WHERE customer_id = 148;

→ Index Scan  ✓
```

Composite index (two columns)

Good when you frequently filter on two columns together.

```
CREATE INDEX idx_rental_customer_date ON rental (customer_id, rental_date);
```

```
-- Both columns in the filter → uses composite index

EXPLAIN ANALYZE

SELECT * FROM rental

WHERE customer_id = 148

      AND rental_date > '2005-06-15';

→ Index Scan using idx_rental_customer_date  ✓
```

Column order matters — the leading column rule

A composite index (A, B) only helps queries that start their filter with A.

```
--  uses the index - customer_id is the leading column

EXPLAIN ANALYZE

SELECT * FROM rental WHERE customer_id = 148;

--  uses the index - both columns present, leading column matches

EXPLAIN ANALYZE

SELECT * FROM rental

WHERE customer_id = 148 AND rental_date > '2005-06-15';

--  does NOT use the composite index - rental_date is not the leading column

EXPLAIN ANALYZE

SELECT * FROM rental WHERE rental_date > '2005-06-15';

→ Seq Scan ← can't use (customer_id, rental_date) index without customer_id
```

The B-Tree is sorted by customer_id first — there's no way to skip straight to rental_date values without knowing customer_id. If you need to filter by rental_date alone frequently, you need a **separate index** on rental_date.

Rule: Put the most selective / most-used filter column first.

Clean up

```
DROP INDEX IF EXISTS idx_rental_customer_id;

DROP INDEX IF EXISTS idx_rental_customer_date;
```

8. EXPLAIN Output — What to Look For

```
Seq Scan on rental (cost=0.00..310.44 rows=30 width=36)

(actual time=0.021..1.842 ms rows=30 loops=1)

Filter: (customer_id = 148)

Rows Removed by Filter: 16014

Planning Time: 0.077 ms
```

Execution Time: 1.874 ms

Field	Meaning
Seq Scan / Index Scan	How rows were found
cost=..	Planner estimate (not ms) — compare relative values
actual time=..	Real milliseconds (startup → total)
rows=..	Actual rows returned
Rows Removed by Filter	Rows scanned but thrown away
Planning Time	Time PostgreSQL spent building the plan
Execution Time	Total actual runtime

Red flags:

- Seq Scan on a large table with a specific WHERE → missing index
- Rows Removed by Filter >> rows= → a lot of waste, index would help
- actual time much higher than cost → stale statistics → run ANALYZE table_name

Good signs:

- Index Scan or Index Only Scan → index is being used
- Index Cond: → filter applied at the index level (most efficient)
- Rows Removed by Filter is 0 or missing → nothing wasted

Key Takeaways

1. **Index** = separate data structure that lets PostgreSQL skip the full table scan
 2. **B-Tree** = default — sorted tree, fast for = , < , > , BETWEEN , prefix LIKE
 3. **EXPLAIN** = shows the plan, doesn't run; **EXPLAIN ANALYZE** = runs and shows real timings
 4. **Seq Scan** = no index — fine for small tables, slow for large
 5. **Index Scan** = index used — direct lookup, fast
 6. **Composite index** (A, B) = helps A alone and A+B together — NOT B alone
 7. **Trade-off** = indexes speed reads, slow writes — don't index everything
-